# B2B Customer Development • Standard Operating Procedure v3.2 (industry-generic edition)

Install Gate + Data Access Layer + Data Triggers + Step 0 + 7 steps + four-axis self-replenishing engine + 10-platform social public search + WhatsApp loop + weekly kit write-back | every stage has an exit and a return path | commands on every step

main flow · fail → back to fix · loop / self-replenishing · to WhatsApp · gate pass / back to Step 0

### ① Install Gate (first thing after install · until all 3 checks pass, every search / send script refuses to run)

**New-install checklist**
one screen of what to fill in now
`▶ onboarding.py start`

→

**Check 1 · Company profile**
industry / offer type / product facts / sender
`▶ init → ask / apply-pack`

→

**Check 2 · Soul Seven Kit health**
missing / empty / unreplaced samples / missing sections
`▶ kit-check 7/7`

→

**Check 3 · User attestation**
AI reads back → you reply 'confirmed'
`▶ check → attest`

→

**All 3 checks pass?**
onboarding.py check

No: fill what is missing (ask · apply-pack · fill_reference_customer_dev.md) → back to Check 1

yes · READY → configure data access first

### ② Data Access Layer (configure right after install · the skill ships no customer database · everything flows into the customer pool)

**SaaS / customs / CRM exports**
drop into inbox → trigger reminds → one import entry
`▶ triggers.py scan`

→

**Own customer DB import (required)**
existing / do-not-contact stay out of cold pool
`▶ data_import.py inspect → import`

→

**Sending channel**
mailbox keys · hosted warm-up · MCP connectors
`▶ mail_setup.py providers · test`

→

**CRM sync**
CSV · HubSpot API · MCP (dry run by default)
`▶ crm_sync.py export · pull`

→

**Data Triggers (daily)**
outbound always human-approved · bounce anomaly auto-pauses mailbox
`▶ triggers.py scan`

into customer pool

trigger queue → Step 0 / matching step

### ♻ Self-replenishing engine (four-axis rotation: Height · Breadth · Depth · Reach)

**Built-in scheduler**
runs once a day at a fixed time; exits on 'pause'
`▶ schedule --at 09:00`

→

**Check customer pool (customer_db)**
sendable cold leads & days of reserve · re-check the day after sending
`▶ replenish`

→

**Red line hit?**
sendable<100 or reserve<30 days

enough · tomorrow → Built-in scheduler

short →

**Four-axis replenishment (never all at once)**
Depth d1 companies → d2 decision makers → d3 deep-dive
Reach regions near → far · Height tiers high → low
Breadth core → wide → max (slowest)
`▶ replenish --emit`

→ today's queries

### Step 0 · Omni-channel Review

win patterns + WA review + last week's kit write-back
MEMORY 'this week's conclusions' = start of this round
`▶ review_success · wa_review review`

→

### Step 1 · Search Candidates (public search only)

keyword matrix · 15 discovery channels (incl. no-signup customs path)
10 social platforms public search + 30 procurement-chain titles
`▶ queries · channels · social`

bottleneck: high dup → new regions/tiers; low named rate → find mailboxes first

candidates →

**★ Three Iron Rules of Search (Search → Research)**

**#1 Check the DB first**
keep only truly new; without --against = warning only
`▶ filter --against`

→

**#2 Funnel diagnosis**
per-stage + cumulative conversion; watch trends
`▶ funnel`

→

**Bottleneck?**
dup in DB>40% · named rate<20%

to pool →

**#3 Pending into pool**
wide entry, strict sending: no named mailbox → pending@
named mailbox required before sending
`▶ email_guesser · dns_validator`

truly new → today's queries

### Step 2 · Background Research (core, never skip)

5 sources + 3 research questions; brand name ≠ need
each conclusion tagged S1 / S2 / S3 + source URL
`▶ research_guide.md`

→

**Enough evidence?**
≥1 S1 or 2 independent S2

S3 only → **Tag need_review → user decides**
AI draws no conclusion; thin info → need_tag=unknown
use the conservative generic line, confirm in follow-up

after user decision →

yes →

### Step 3 · Need Positioning (dual tags)

need_tag → which line · solution_tag → which hook
violates positioning.tag_rules → no send, to human
`▶ conflict-check`

→

### Step 4 · Write Content

recipient's native language · ≤120 words · plain text
S1/S2 facts only · follow-ups change angle, add news
`▶ email_templates.md`

any fail → fix, re-check

→

**Three pre-send checks**
mailbox ✓ dedup ✓ notes ✓

all pass →

### Step 5 · Send

DRY_RUN preview → user approval → named decision maker
recipient's local business window · ≤200/day · 30s interval
`▶ decision_maker_sender.py`

outside window → **Outside local window → defer**
report deferred_window, no history row
`▶ send_window.py table / check`
next window → Step 5

→

### Step 6 · Follow-up Management

24–48h bounce / reply scan (read-only mailbox, DRY_RUN first)
social DMs (manual) 1-3-7-21; one channel per day
`▶ imap_scanner.py · cooldown`

→

**Bounce**
mark_bounced: blacklist + stop for good
→ back to pool for replenishment
`▶ mark_bounced`

bounce → replenish pool

**Reply → human follow-up**
quotes / lead times / replies need user approval
explicit refusal → add_blacklist, stop all channels
`▶ mark_replied`

**No reply → 5-level cooldown**
followup_count = e-mails sent − 1
`▶ csv_ops.py cooldown`

→

**followup_count?**
COOLDOWN_RULES

0/1/2 (1–3 sent): wait 3 / 3 / 10 days → new-angle follow-up → Step 4

=3 (4 e-mails sent) →

**Stop generic mailboxes → LinkedIn / personal only**
manual · social 1-3-7-21; Day 21 no reply → check WA signals

Day 21 no reply + WA signal →

≥4 (5+ e-mails sent): stop e-mail → WhatsApp →

**Order / won**
AI proposes evidenced pain-point candidates only
stored only after user confirmation → Step 8

won → Step 8

**Safety & compliance baseline**
credentials via env vars only · 465/587 encrypted
send / bounce scan / DNS / WA all have dry run
generic mailbox block · bounce = permanent blacklist · read-only
social scripts emit public queries only; login / DMs manual

back to Step 0 · next round

### Step 7 · WhatsApp Trigger + bulk send + dedicated review

**WA trigger signals (any one)**
≥3 e-mails no reply · bounced, no alternative
high value + public WA · customer gave number

→

**List → bulk-send input**
wa_targets.json / CSV → contacts JSON
`▶ wa_bulk_sender.py convert`

→

**Validate + read-only preview**
numbers / duplicates / empty / placeholders
fail → fix list or template, convert again
`▶ validate → preview`

→

**Test-number send**
only to your own test number
`▶ harness`

→

**One-by-one bulk send (anti-ban rules)**
screenshot check · 8–15s interval · 200 → STOP
`▶ send · progress`

→

**Screenshot audit**
undelivered / wrong person → check individually
`▶ audit`

→

**Write back + won channel**
wa_status=sent · add_success --won_channel wa

→

**WA dedicated review (read-only)**
funnel + by region / need / hook
`▶ wa_review review / funnel`

next → WA trigger signals

merge → Step 8

### Step 8 · Win Review + Kit Write-back (closing the loop · weekly)

**Store success case**
pain point = user-confirmed version
`▶ add_success --won_channel`

→

**Omni-channel review**
by region / need_tag / hook_type / channel
`▶ review_success · email_dashboard · wa_review`

attribute →

**Win attribution (won_channel)**
e-mail vs WA: which channel wins more
→ weight channels / regions / hooks next round
customs, social etc. counted by source_channel

→

**Weekly kit write-back**
wrong / missed / overreach → attribute to one file
fix the rule, not the sentence; update HEARTBEAT
`▶ soul_kit/weekly_writeback.md`

→

**Re-check → next round**
write conclusions into MEMORY
`▶ onboarding.py kit-check`

→ Step 0

### Data layer (throughout · send records only via csv_ops.py)

customer_db · sent_history · blacklist · wa_contacts · success_cases · pipeline · mailbox_usage · crm_sync_log | before: check / cross_brand_check · after: add_sent · bounce: mark_bounced
Iron rule: scripts never edit customer data by themselves; pain points, tags, sending, CRM push and quotes are user decisions; keys via env vars only
`▶ consistency_audit.py`

Loop check: the exit / return path of every node is listed in references/sop_flowchart.md §4. The generic template contains no company, contact or phone number; all sample values are filled in by the user at install. Platform actions that need login (connection requests, DMs, joining groups) are done manually by the user only.